pcb_homogenization

PCB copper-layout homogenization and solder-joint thermal-cycling simulation · module overview with example results

What it does

A 2-layer PCB layout (Gerber) is rasterized into copper/FR4 maps, coarse-grained by a quadtree with per-leaf Voigt–Reuss homogenization, and mapped onto an ANSYS MAPDL board model. A partner-supplied CR0805 resistor is placed on the board with SAC305 solder, the assembly is driven through a reflow and thermal-cycling profile with creep, and the solder creep strain is post-processed per cycle as input for a fatigue-life law.

Pipeline

1	`import_real_board.py`	Gerber → sample folder
2	`discretize.py`	copper rasters → `mesh/F_Cu_map.npy`, `B_Cu_map.npy`
3	`quadtree.py`	Voigt–Reuss leaves → `mesh/quadtree.json`
4	`simulate_pyansys_assembly.py`	MAPDL solve → `results/pyansys_assembly.rst`, `thermal_profile.yaml/.png`
5	`zone_creep_strain.py`	critical-zone creep strain → `results/zone_creep_strain.json`
6	`plot_cycle_creep_increment.py`	$\Delta\varepsilon$ per cycle → `results/cycle_creep_increment.json`

Example used throughout

Board `rv3032_example` (20 mm × 15 mm, Cu 35 µm / FR4 1.53 mm), solved with the shortened example profile `configs/thermal_profile_example.yaml` (cool-down from the stress-free 180 °C to 22 °C with a 300 s hold, then one –40–125 °C cycle), a 20 × 20 board mesh and 8 cores: about 8.5 min wall time, 60 substeps, 0.6 GB result file.

Module map

`import_real_board.py`	Imports real F_Cu/B_Cu Gerbers, shifts them to a (0,0) origin, writes `metadata.json`.
`discretize.py`	Parses RS-274X Gerber copper layers and rasterizes them to binary maps.
`homogenization.py`	Material loading, Voigt/Reuss bounds.
`quadtree.py`	2:1-balanced (or leaf-budget) quadtree with per-leaf homogenized properties.
`plot_board_model.py`	Top view of the simulated board (projected or homogenized), read from the `.rst`.
`solder_reliability.py`	Tracked component pads; quick analytical CTE-mismatch / Coffin–Manson estimate.
`simulate_pyansys_benchmark.py`	Board-only MAPDL model: structured hex mesh painted from the rasters.
`component_import.py`	Imports the partner `.cdb` component, remaps material IDs, MPC bonded contact.
`simulate_pyansys_assembly.py`	Board + CR0805 + SAC305 assembly, thermal profile, creep solve.
`thermal_profile.py`	Loads, plots and applies the time/temperature YAML (time table, one load step per segment); cycle boundaries.
`plot_creep_strain_history.py`	Maximum/mean solder creep strain vs. time.
`zone_creep_strain.py`	Critical-zone volume-averaged creep strain (Schwerz / Darveaux method).
`plot_cycle_creep_increment.py`	Per-cycle creep-strain increment $\Delta\varepsilon$ for the fatigue law.
`plot_component_materials.py`	3-D render of the assembly coloured by material.

1 Board input

`import_real_board.py` converts an externally authored board into the common sample layout (`metadata.json` + `gerber/`): it copies the top and bottom copper Gerbers, shifts them so the board starts at (0,0), and records the stackup. Only the two outer copper layers and the dielectric core are kept.

2 Discretization

`discretize.py` parses each RS-274X copper layer (pads, traces, regions) and rasterizes it onto a uniform grid (`--resolution 40` px/mm → 800 × 600 px here; default 10) and writes `F_Cu_map.npy`, `B_Cu_map.npy` and `discretization.json`. The top layer carries the signal traces and the resistor pads (R1, blue markers); the bottom layer is a ground pour with clearances.

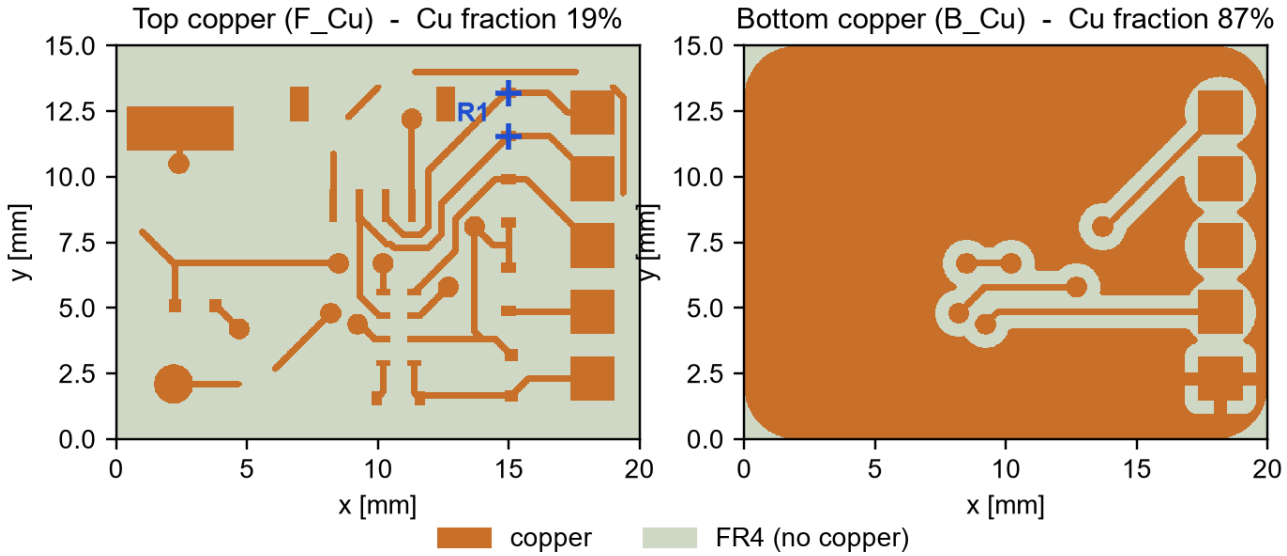


Figure 1 – Input copper layers of `rv3032_example` after rasterization. Blue crosses: R1 pad centres used for component placement (`solder_reliability.py`).

3 Homogenization (quadtree)

`quadtree.py` splits the board wherever a leaf straddles a copper/FR4 boundary (pure leaves are already exact) and enforces 2:1 balance, or alternatively stops at a leaf budget (`--max-leaves`). Each leaf gets Voigt (in-plane) and Reuss (out-of-plane) stiffness and CTE from its top/bottom copper fractions (`homogenization.voigt_reuss_bounds()`). Output: `mesh/quadtree.json`.

The MAPDL board model does not use these homogenized properties: it projects the layout directly onto its elements (Section 4). `mesh/quadtree.json` currently feeds only the quick analytical CTE-mismatch estimate in `solder_reliability.py`.

4 Trace mapping onto the MAPDL board mesh

`simulate_pyansys_benchmark.py` builds a structured hex mesh of the board (bottom Cu / FR4 core / top Cu; nodes and elements created directly via PyMAPDL) and paints each element copper or FR4 by sampling the raster at the element centre. The mesh density (`--nx`, `--ny`) therefore decides how much of the layout survives: at 50 × 50 (0.4 mm × 0.3 mm elements) the traces are still recognizable, at 20 × 20 most thin traces vanish and only pads and wide features remain.

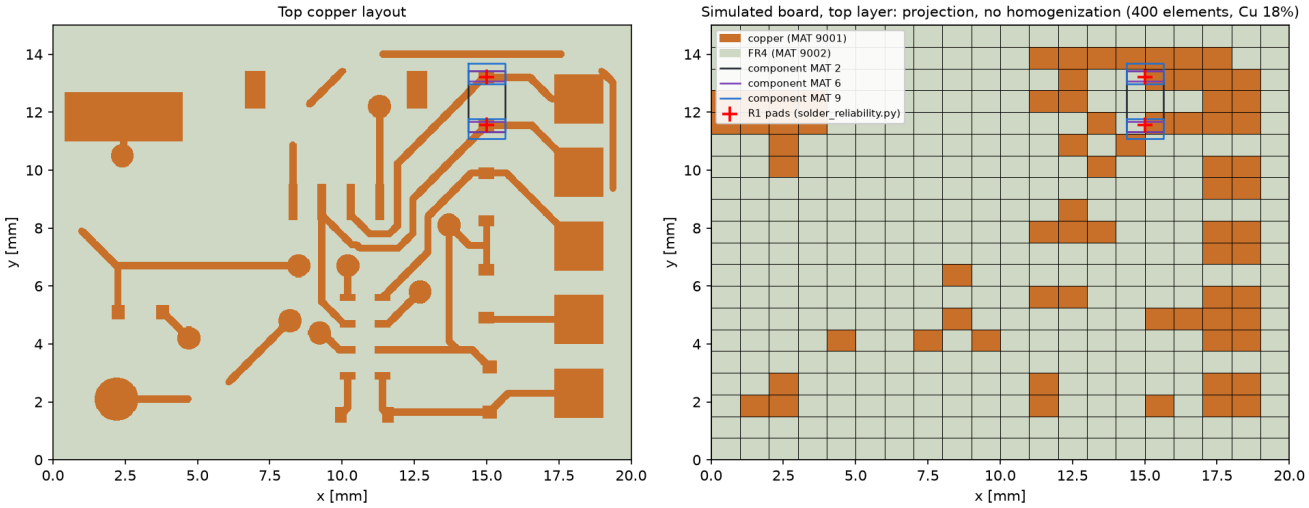


Figure 2 – Top copper layout (left) and the top copper layer of the board as it was simulated (right), read back from the result file by `plot_board_model.py`: 20 × 20 elements, each pure copper or pure FR4 (no homogenization). Outlines: the simulated CR0805 body (MAT 2), terminations (MAT 6) and solder (MAT 9); red crosses: the R1 pad centres used for placement. For a homogenized board the right panel shows the per-element in-plane modulus as a colormap instead.

5 Component and solder

`component_import.py` reads the partner-supplied CR0805 archive (`DemonstratorB_CR0805_Export.cdb`) into the running MAPDL session without remeshing it. Board materials are moved to MAT 9001/9002 first so they cannot collide with the archive's own material IDs, and the SAC305 solder law (elastic + creep) is applied from `SAC305_MikroVal.mac`. `simulate_pyansys_assembly.py` places the component at the midpoint of the two R1 pads (`PAD_1_XY_MM`/`PAD_2_XY_MM` in `solder_reliability.py`, taken from the Gerber), turns it onto the pad axis (here 90 °), drops the solder onto the board's top surface and connects it via MPC bonded contact, reusing the TARGE170/CONTA174 template from the archive. R1 is an 0603 footprint (0.55 mm × 0.40 mm pads, 1.65 mm apart) while the archive is a CR0805, so the component overhangs the pads; Fig. 2 shows its outline on the layout.

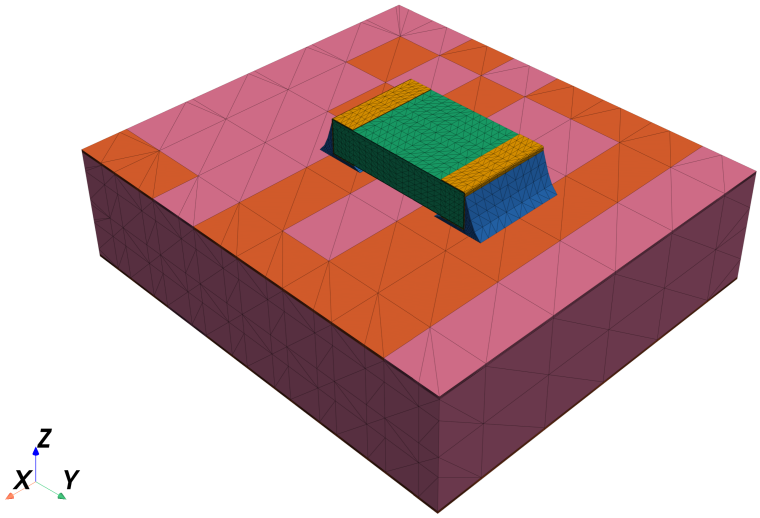


Figure 3 – Assembly around R1, coloured by material (`plot_component_materials.py`): ceramic body (green, MAT 2), nickel terminations (yellow, MAT 6), SAC305 solder fillets (blue, MAT 9), board copper (orange, MAT 9001) and FR4 (pink, MAT 9002) at 20 × 20 board resolution.

6 Thermal load

`thermal_profile.py` reads a list of (time, temperature) points and applies them as a uniform temperature field from a MAPDL time table, so every substep sees the exact profile temperature; each segment is solved as its own load step. TREF is 180 °C, the upper end of the solder's material data, and the example profile starts there, so the solve begins from an undeformed, stress-free state. The profile is plotted before the solve (`results/thermal_profile.png`, or standalone via `python thermal_profile.py <profile.yaml>`) and saved next to the results; its cold-hold starts are the cycle boundaries used below.

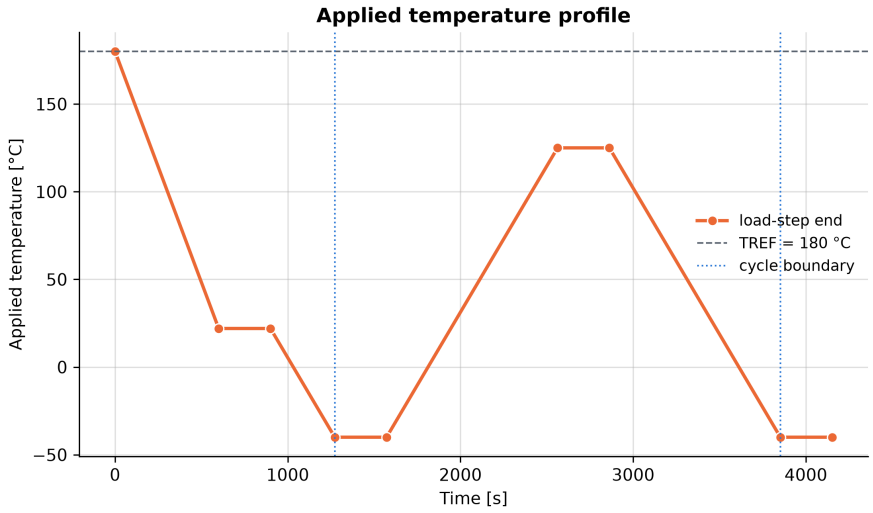


Figure 4 – Applied load of the example profile: cool-down from 180 °C in 600 s, 300 s hold at room temperature, then one thermal cycle with 300 s holds. Markers are load-step ends; dotted lines are the cycle boundaries.

7 Creep post-processing

All post-processing reads the `.rst` directly (ansys-mapdl-reader); no MAPDL session or licence is needed. `zone_creep_strain.py` selects the critical solder sub-volume (here 25 % of the solder volume with the highest accumulated creep strain, NL,CREQ) and volume-averages its creep strain over time, following R. Schwerz's (Fraunhofer IKTS) APDL macro and Darveaux's method.

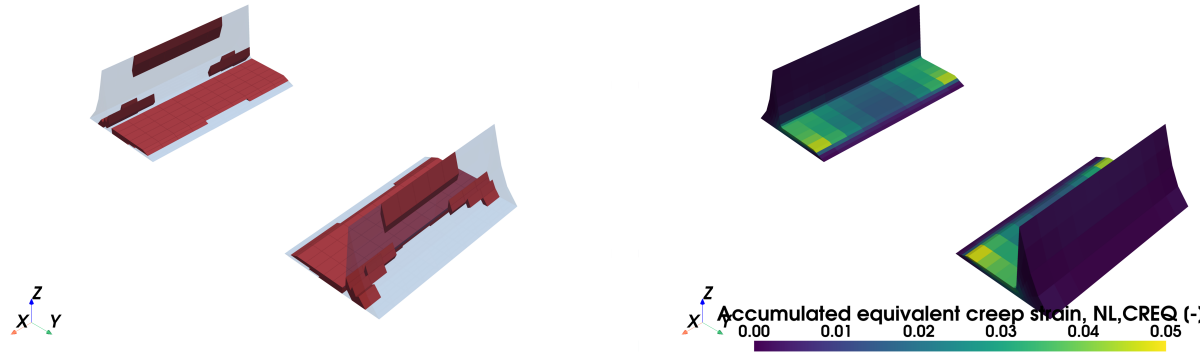


Figure 5 – Critical zone: 346 of 1040 solder elements (left) and accumulated creep strain at the end of the run (right), both from `zone_creep_strain.py`.

`plot_cycle_creep_increment.py` cuts the zone-averaged history at the thermal-cycle boundaries and reports the creep-strain increment per cycle, the $\Delta\varepsilon$ that enters a Coffin–Manson-type fatigue law. Plotting every cycle shows whether the response has stabilized (shakedown) before a value is picked.

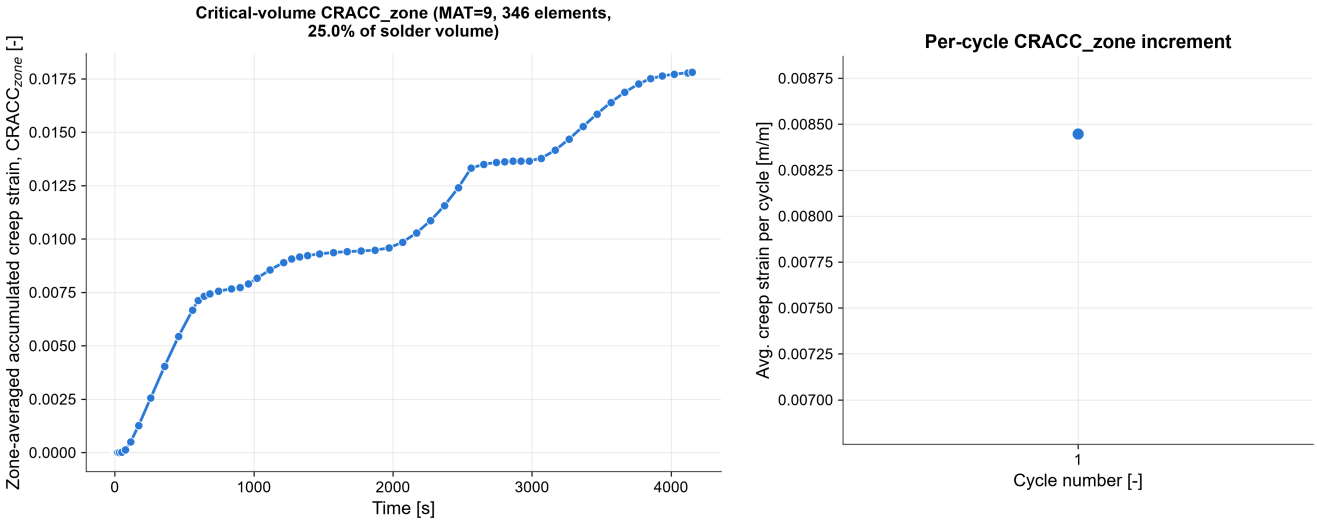


Figure 6 – Zone-averaged accumulated creep strain vs. time (left) and its per-cycle increment (right): $\Delta\varepsilon = 8.4 \times 10^{-3}$ for the single cycle of the example run.

The example run demonstrates the workflow. With one cycle, short holds and a coarse 20 × 20 board mesh that loses most traces under the joint (Fig. 2), its $\Delta\varepsilon$ is not a fatigue value: use the full profile, several cycles and the default mesh for that.

8 Board-model comparison

`plot_comparison_creep.py` overlays the zone-averaged creep history and per-cycle $\Delta\varepsilon$ of several `simulate_pyansys_assembly.py` runs on the same board, read back from their saved `results/` JSON (no `.rst` access needed). `--board-model homogenized` gives each board element the orthotropic Voigt–Reuss material of its quadtree leaf, smeared through the full thickness; `homogenized_zlayers` instead resolves the two copper layers separately – each gets its own per-leaf material from `phi_bot`/`phi_top` alone, and the FR4 core in between stays plain FR4. Both need `mesh/quadtree.json` from `quadtree.py`.

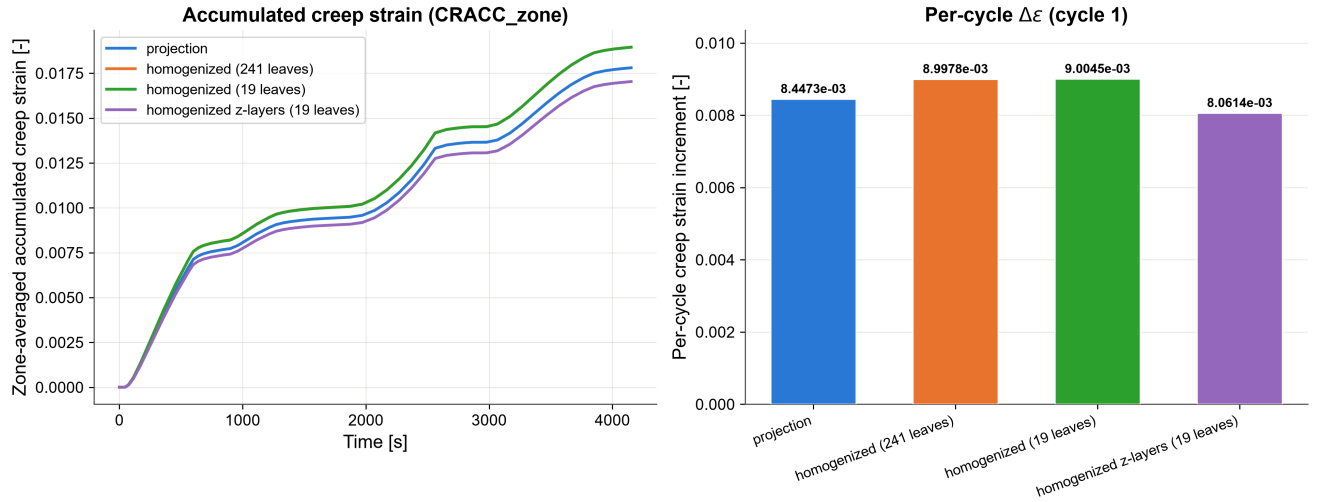


Figure 7 – Zone-averaged accumulated creep strain vs. time (left) and the per-cycle increment (right) across four board models of the same `rv3032_example` board: the directly projected copper/FR4 raster, full-resolution homogenization (241 leaves), the coarser 19-leaf quadtree, and the same 19-leaf quadtree with the two copper layers homogenized separately (`homogenized_zlayers`). The coarse quadtree tracks the full-resolution homogenization closely ($\Delta\varepsilon = 9.00 \times 10^{-3}$ vs. 9.00×10^{-3}, both ∼6.5 % above the projection's 8.45×10^{-3}), while resolving the copper layers separately pulls $\Delta\varepsilon$ below the projection instead (8.06×10^{-3}, $-4.6\,\%$) – smearing both layers into one orthotropic material over- or under-stiffens the joint differently than keeping them apart.